

Practical Lab: From Autoencoders to Variational Autoencoders

Generative Models – M2 Computer Science

Duration: 2 hours, including setup and a short report
Dataset: MNIST
Hardware: one small GPU per student

Learning goals

By the end of this lab, you should be able to:

1. train and use a simple autoencoder (AE);
2. visualize a 2D latent representation;
3. explain why a good autoencoder is **not automatically a good generative model**;
4. fit a Gaussian mixture model (GMM) to the latent codes of an autoencoder and sample from it;
5. implement the three key ingredients of a variational autoencoder (VAE):
 - a Gaussian encoder,
 - the reparameterization trick,
 - the reconstruction + KL objective;
6. generate new MNIST digits and explore the learned latent space.

The lab is deliberately small. Most of the PyTorch infrastructure is already written. Your task is to complete a few important lines and interpret the results.

1. Setup

Clone the repository provided by the instructor, then create the environment:

```
conda env create -f environment.yml
conda activate genmodels-vae
```

Check that PyTorch, CUDA, and the dataset are available:

```
python check_install.py
```

If MNIST is stored in a shared directory on the cluster, use:

```
python check_install.py --data-dir /path/to/shared/MNIST
```

The expected final lines are similar to:

```
PyTorch: OK
CUDA: OK
MNIST: OK
Everything is ready!
```

Important. The practical work is in `student.py`. Search for `TODO`. You should not need to modify `run_lab.py` or `utils.py`.

You can list all `TODO`s with:

```
grep -n "TODO" student.py
```

2. Autoencoder: good reconstruction does not imply good generation

An autoencoder contains an encoder E_ϕ and a decoder D_θ :

$$z = E_\phi(x), \quad \hat{x} = D_\theta(z).$$

We train it to reconstruct the input:

$$\min_{\theta, \phi} \sum_i \|x_i - D_\theta(E_\phi(x_i))\|^2.$$

In this lab the latent variable has dimension **2**, so we can visualize it directly.

Task 1 – Complete the autoencoder forward pass

Open `student.py` and complete **TODO 1**.

Then train the autoencoder:

```
python run_lab.py ae
```

The script creates:

- `outputs/ae_reconstructions.png`
- `outputs/ae_latent.png`
- `outputs/ae_prior_samples.png`
- `outputs/ae.pt`

Look at all three figures before continuing.

Question 1. The autoencoder can reconstruct digits, but samples obtained by drawing

$$z \sim \mathcal{N}(0, I_2)$$

and decoding them are often poor. Why is this not surprising?

3. Learning a distribution in the latent space with a GMM

The autoencoder has learned a set of latent codes $z_i = E_\phi(x_i)$, but it has **not** learned a probability distribution from which we can sample latent variables.

A simple idea is therefore:

1. encode the MNIST training images;
2. fit a Gaussian mixture model to their 2D latent codes;
3. sample new latent variables from the GMM;
4. decode them with the already-trained decoder.

A mixture of K Gaussians has density

$$p(z) = \sum_{k=1}^K \pi_k \mathcal{N}(z; \mu_k, \Sigma_k).$$

We use `sklearn.mixture.GaussianMixture`, which performs the EM optimization for us.

Task 2 – Fit and sample the GMM

Complete **TODO 2** and **TODO 3** in `student.py`.

Then run:

```
python run_lab.py gmm
```

This creates:

- `outputs/ae_latent_gmm.png`
- `outputs/ae_gmm_samples.png`

Question 2. Compare `ae_prior_samples.png` and `ae_gmm_samples.png`. Why should sampling from the fitted GMM work better than sampling from $\mathcal{N}(0, I_2)$?

Question 3. After fitting the GMM, we can generate according to

$$z \sim p_{\text{GMM}}(z), \quad x = D_{\theta}(z).$$

In what sense is **GMM + decoder** now a generative model?

4. Variational autoencoder

Instead of fitting a distribution to the latent codes **after** training, a VAE learns a latent representation that is compatible with a chosen prior during training.

We use the generative model from the course:

$$z \sim \mathcal{N}(0, I_2), \quad x | z \sim \mathcal{N}(D_{\theta}(z), I).$$

The exact posterior $p_{\theta}(z | x)$ is difficult to compute, so the encoder represents an approximate Gaussian posterior

$$q_{\phi}(z | x) = \mathcal{N}(\mu_{\phi}(x), \text{diag}(\sigma_{\phi}(x)^2)).$$

The network will output `mu` and `logvar = log(sigma^2)`.

Task 3 – Gaussian encoder

Complete **TODO 4** so that the encoder returns the mean and log-variance of the approximate posterior.

Task 4 – Reparameterization trick

To backpropagate through a random latent variable, write

$$\varepsilon \sim \mathcal{N}(0, I), \quad z = \mu + \sigma \odot \varepsilon.$$

Complete **TODO 5**.

Useful PyTorch functions include:

```
torch.exp(...)
torch.randn_like(...)
```

Remember that the network outputs `logvar = log(sigma^2)`.

Task 5 – VAE objective

For a diagonal Gaussian posterior and a standard Gaussian prior,

$$D_{\text{KL}}(\mathcal{N}(\mu, \text{diag}(\sigma^2)) \parallel \mathcal{N}(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2).$$

Because the decoder likelihood is Gaussian, the reconstruction contribution is proportional to a squared reconstruction error.

We minimize

$$\mathcal{J} = \underbrace{\|x - D_\theta(z)\|^2}_{\text{reconstruction}} + \beta \underbrace{D_{\text{KL}}(q_\phi(z | x) \parallel \mathcal{N}(0, I))}_{\text{regularization}}.$$

Complete **TODO 6** and **TODO 7** in `student.py`.

Then train the VAE:

```
python run_lab.py vae
```

The script creates:

- `outputs/vae_reconstructions.png`
- `outputs/vae_latent.png`
- `outputs/vae_prior_samples.png`
- `outputs/vae_training_curves.png`
- `outputs/vae.pt`

Question 4. What is the role of the KL term? Compare the AE and VAE latent spaces.

Question 5. Why do we use the reparameterization

$$z = \mu + \sigma \varepsilon$$

instead of simply writing “sample $z \sim q_\phi(z | x)$ ” inside the network?

5. Explore the VAE latent space

Now use the trained VAE as a generative model.

Run:

```
python run_lab.py explore
```

This produces:

- `outputs/vae_latent_grid.png`: decode a regular grid of points in latent space;
- `outputs/vae_interpolation.png`: encode two MNIST images and interpolate between their latent means.

Question 6. Look at the latent grid and the interpolation. Describe one qualitative property of the latent representation that is useful for generation.

6. Optional extension – what does beta do?

If you finish early, retrain the VAE with a different weight on the KL term:

```
python run_lab.py vae --beta 0 --tag beta0
python run_lab.py vae --beta 5 --tag beta5
```

You can add `-tag beta0`, `-tag beta5`, etc. to keep several runs without overwriting previous figures.

Compare reconstruction quality, latent organization, and samples from the prior.

Optional question. What trade-off do you observe when β is increased?

7. Report

Submit a **short report (maximum 2 pages)**. A LaTeX template is provided in `report_template.tex`.

Your report should contain:

1. one AE reconstruction figure and a short comment;
2. a comparison of AE samples from $\mathcal{N}(0, I)$ and from the fitted GMM;
3. one VAE latent-space figure and one VAE generation figure;
4. concise answers to Questions 1–6.

We are interested in your **interpretation**, not in a description of every line of code.

Useful commands

Fast debugging run on a small subset:

```
python run_lab.py ae --quick
python run_lab.py vae --quick
```

Choose CPU explicitly:

```
python run_lab.py ae --device cpu
```

Choose another data directory:

```
python run_lab.py ae --data-dir /path/to/MNIST
```

Clean generated outputs:

```
rm -rf outputs
```

Good luck.